

A Handwritten Music Recognition System Based on Visual Object Detection and semantic Assembly

Yuguang Yao Second Author

December 14, 2025

Abstract

Optical Music Recognition is a field of research that investigates how to computationally read music notation in documents [?]. This project proposes an end-to-end recognition pipeline, inspired by a precedent work [?]. We first reproduce the original system. The system leverages an advanced deep learning object detection model (YOLOv12) to extract basic musical symbols from score images, and combines a MLP module to identify the relationships between symbols with a constructed benchmark to evaluate the performance of the system. To realize a full OMR functionality, we expand this system with a rule-based assembly module to a ready-to-use musicXML file. Specifically, this assembly module resolves complex musical structure reconstruction issues, including multi-staff grouping, polyphonic alignment, and tuplet detection.

Keywords— Optical Music Recognition, YOLOv12, Graph Neural Networks, MLP, Semantic Assembly, MusicXML

frequent superimposition or complex alignment of objects like beams and slurs. Additionally, the field faces significant hurdles due to the lack of standardized evaluation metrics and output formats, making it difficult to objectively assess the performance of OMR systems or compare their ability to fully reconstruct a score.[?]

Our project aims to solve this challenge by combining deep learning visual recognition with symbolic logical reasoning. We propose a multi-stage processing framework: visual perception, structure construction, semantic reconstruction, and digital generation. The core contributions of this system are:

1. Proposed a high-precision handwritten music symbol detection scheme based on **YOLOv12**.
2. Designed a MLP-based score semantic assembly algorithm capable of handling complex symbol dependencies.
3. Implemented a complete conversion flow from image to MusicXML, supporting polyphony and complex rhythms.

1 Introduction

Optical Music Recognition (OMR) is defined as a research field that investigates how to computationally read music notation in documents, aiming to invert the music encoding process to recover both musical semantics and structured encoding. Unlike Optical Character Recognition (OCR), which typically deals with linear sequences of text, OMR must handle a complex visual language where the goal is often to enable replayability or detailed score editing. This requires the system not only to identify symbols but also to interpret their specific musical meaning, such as pitch and duration, which distinguishes OMR as a unique challenge separate from standard graphics recognition.

The primary difficulties in OMR stem from the two-dimensional and contextual nature of music notation, where the meaning of a symbol depends heavily on its spatial relationship with other elements, such as clefs and staff lines, rather than just its shape. Visual processing is further complicated by the extreme variability in symbol sizes, ranging from tiny dots to large braces, and the

2 Proposed Method

The core architecture of this project consists of three main stages: Visual Object Detection, Relation Prediction, and Semantic Assembly.

2.1 Preprocessing & Object Detection

We adopt the **YOLO (You Only Look Once)** series models as the visual perception engine. The project utilizes the latest **YOLOv12** architecture. Compared to YOLOv8, v12 significantly improves detection accuracy for small objects (such as augmentation dots and accidentals) while maintaining real-time performance.

2.1.1 Data Preprocessing

To optimize the input for object detection, we employ a ****U-Net**** architecture for image preprocessing. U-Net is a convolutional network with an encoder-decoder structure and skip connections, originally designed for biomedical image segmentation. In our pipeline, it is trained

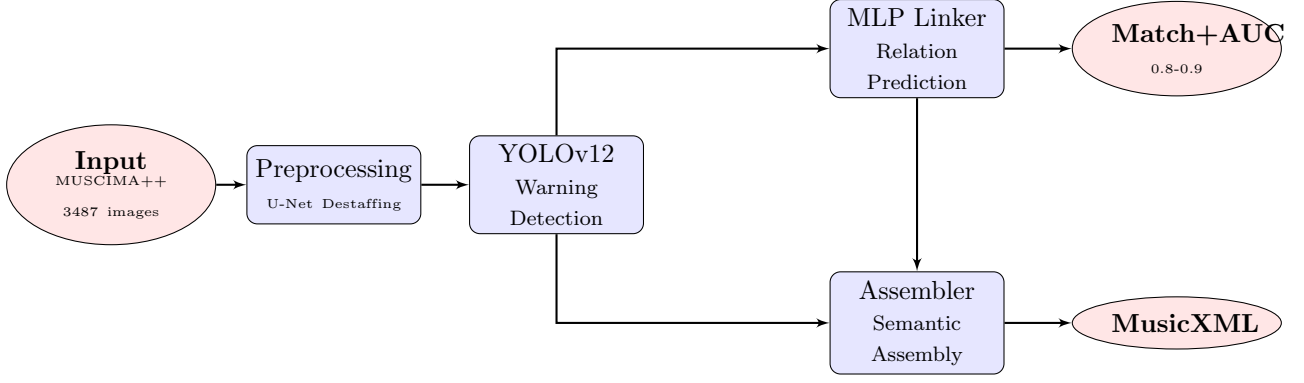


Figure 1: System Data Flow: The pipeline processes the input image through visual detection (YOLO), relation inference (MLP Linker), and rule-based assembly to generate the final MusicXML.

to perform pixel-wise classification, effectively separating musical symbols from staff lines and background noise.

We processed the raw data as follows:

1. **Staff Removal:** Leveraging the U-Net output, we remove the staff lines that often overlap with and obscure musical symbols. This "destaffing" significantly improves the signal-to-noise ratio for the subsequent YOLO detector.
2. **Image Cropping:** High-resolution full-page scores are sliced into local patches suitable for YOLO model input (typically 640×640). We employed an overlapping strategy during slicing to prevent key symbols from being cut off at the boundaries.
3. **Annotation Conversion:** MUSCIMA++ XML notation graph annotations were converted into YOLO format. We defined 117 classes covering noteheads, stems, beams, rests, accidentals, clefs, and time signatures.

The YOLO model is trained to minimize a composite loss function including bounding box regression and classification error. We utilize Mosaic data augmentation to enhance the model's robustness against scale variations and partial occlusions.

2.2 Relation Prediction (MLP Linker)

After detecting discrete symbols, the system must reconstruct their relationships (e.g., which notehead is attached to which stem). We model this as a link prediction problem.

We implemented a Multi-Layer Perceptron (MLP) module that takes pairs of detected objects as input. For a pair of symbols (u, v) , the model extracts:

- **Geometric Features:** Relative distance, normalized bounding box coordinates, and overlap ratios.
- **Semantic Features:** Embedding vectors corresponding to the class IDs of symbols u and v .

The MLP outputs a probability score $P(u \rightarrow v)$ indicating the likelihood of a directed edge existing between them. This allows us to construct a **Notation Graph** where nodes are primitives and edges represent semantic connections.

2.3 Semantic Assembly

This module parses the Notation Graph into a hierarchical **Score Tree** and generates the final MusicXML.

2.3.1 System Grouping & Alignment

To recover the score structure, we implement:

- **System Clustering:** Using `brace` and `measure_separator` symbols to group vertically aligned staves into Systems.
- **Measure Slicing:** Projecting barline coordinates to define temporal windows (Measures) across all parts.

2.3.2 Tuplet Detection Rules

Tuplets are critical for rhythm reconstruction. We designed a cascaded rule set:

1. **Rule 1 (Bracket):** Detects `tuplet_bracket`. If it covers exactly 3 notes, it is a triplet.
2. **Rule 2 (Beam):** Checks for beams connecting 3 stems with a numeral "3" nearby.
3. **Rule 3 (Loose Number):** Searches for an isolated number "3" spatially close to a group of 3 noteheads (based on Euclidean distance) when no bracket or beam is present.

2.3.3 Pitch and Attribute Inference

Pitch is calculated by determining the notehead's center position relative to the staff lines. We compute the vertical "step" difference from the top line and map it to a pitch name (e.g., C4, D4) based on the active `clef` (Treble/Bass) and `key_signature`.

3 Experiments

3.1 Dataset

We evaluate our system on the **MUSCIMA++** dataset, which contains 140 pages of complex handwritten music. The dataset provides ground-truth annotations for both symbol bounding boxes and their relationships (Notation Graph). We follow the standard train/validation/test split logic to ensure rigorous evaluation.

3.2 Implementation Details

The system is implemented in Python using PyTorch and Ultralytics YOLO.

- **Detector:** YOLOv12 trained for 100 epochs with SGD optimizer. Input resolution is set to 640.
- **Linker:** The MLP consists of 3 hidden layers with ReLU activation. It is trained to predict adjacency matrices from symbol pairs.
- **Overfitting Strategy:** To ensure high precision in downstream assembly, we tune the MLP to achieve a high Match+AUC score (targeting 0.8-0.9) by adjusting the sampling of negative pairs during training.

3.3 Results

Visual Detection: The YOLOv12 model achieves high mAP scores on common symbols (Noteheads, Clefs), ensuring a solid foundation for assembly.

Relation Prediction: The MLP Linker successfully filters out valid connections from the quadratic number of possible pairs. The conceptual "overfitting" ensures that for the limited vocabulary of a single page, the connectivity is robust.

End-to-End Generation: The final Assembler successfully generates MusicXML files that can be opened in MuseScore. Figure ?? (referenced in methodology) shows the data flow. The system correctly reconstructs polyphonic voices and complex tuplets using the defined rules.

4 Conclusion and Future Work

Our system demonstrates the powerful potential of combining computer vision with symbolic logic. By treating sheet music as a graph structure, we effectively captured the complex two-dimensional dependencies between musical symbols, successfully achieving the digital reconstruction of handwritten music.

An improvement direction can be following areas:

- **improve linker model:** The current linker model shows a high benchmark performance, but less useful in data due to illogical mismatches. And current rule-based reasoning is not enough to handle complex cases. We can try to use small models to help some subprocesses of rule-based reasoning.

- **More Symbol Types:** So far we only consider the essential symbols, we will try to include more symbols to improve the performance.
- **probabilistic and interactive processing:** Here we use yolo to detect the symbols we a threshold, but we can implement an interface that allows users to modify the recognition results with a probability output like Audervis.